Chapter 3: Processes





Chapter 3: Processes

- Process Concept
- Process Scheduling
- Operations on Processes
- Cooperating Processes
- Interprocess Communication
- Communication in Client-Server Systems





Process Concept

An operating system executes a variety of programs:

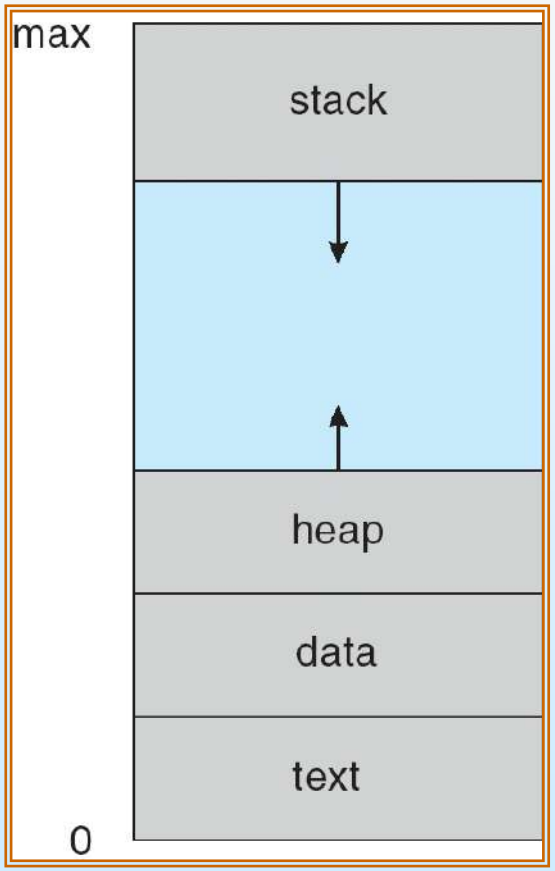
- Batch system – jobs
  - A number of jobs sent for execution, OS transfer the control from one job to the next, the user do not interact directly with the OS, the user just prepare the data and program for his job then send it with other jobs to the OS for execution.
- Time-shared systems – user programs or tasks executed simultaneously using CPU scheduling.

- Process – a program in execution; process execution must progress in sequential fashion
- A process includes:
  - program counter (to indicate the current activity).
  - Stack (contains temporary data (such as local variables, function parameters, etc..)
  - data section (contains global variables)
  - Heap: it is a memory that is dynamically allocated during process run time.





Process in Memory







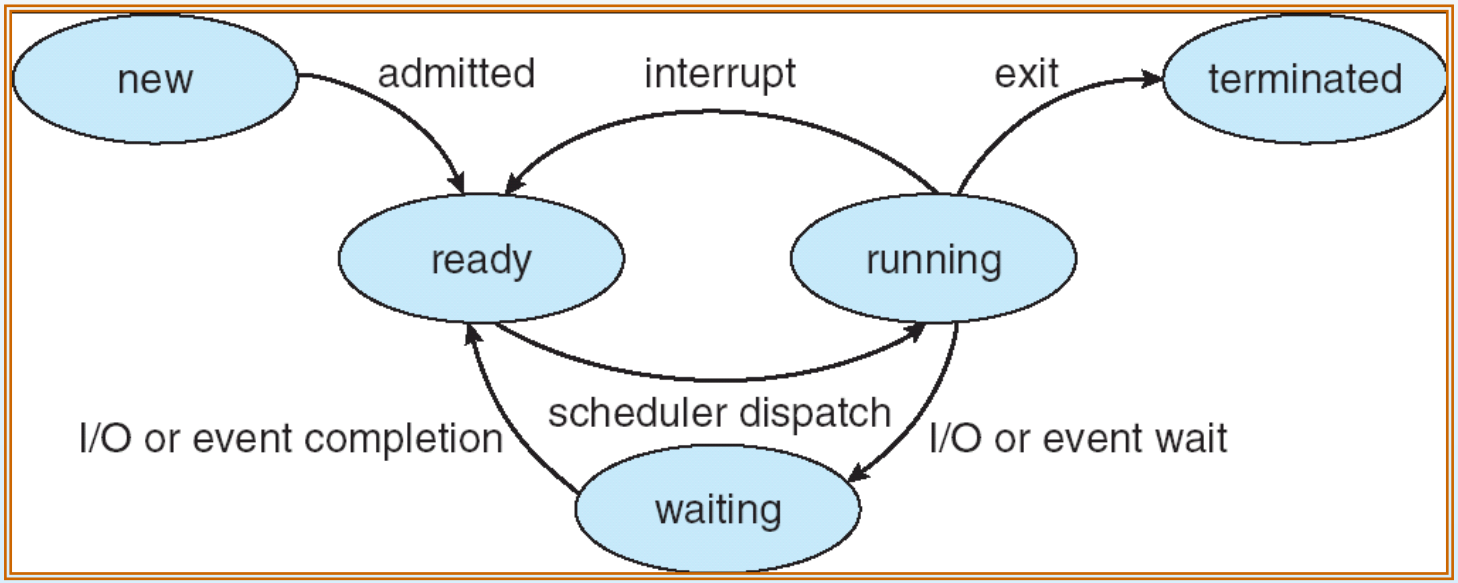
Process State

- As a process executes, it changes *state*
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution





Diagram of Process State







Process Control Block (PCB)

- Each process is represented in the OS by PCB also known as task control block.
- Information associated with each process
- Process state (ready, running, etc…)
- Program counter (indicates the address of the next instruction to be executed for this process).
- CPU registers (because the registers different in type and number, their information (related to type and number) should be stored for the current process when an interrupt occurs, to allow the process to be continued correctly afterward (see slide 9)).





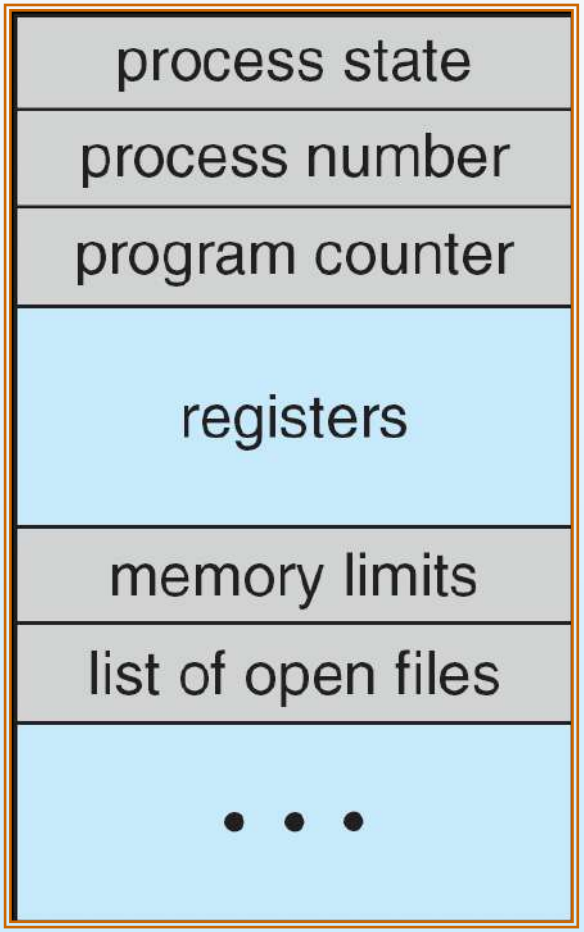
Process Control Block (PCB)

- CPU scheduling information (it contains all the parameters related to the CPU scheduling such as process priority. (chapter 5 will discuss this in more details).
- Memory-management information (such as the value (address) of the base and limit registers in MM. (chapter 8 will introduce more details about this subject)).
- Accounting information (includes information about the amount of CPU used, process numbers, time limits, etc..)
- I/O status information (includes information about the list of I/O devices allocated for the process, a list of open files, etc..)





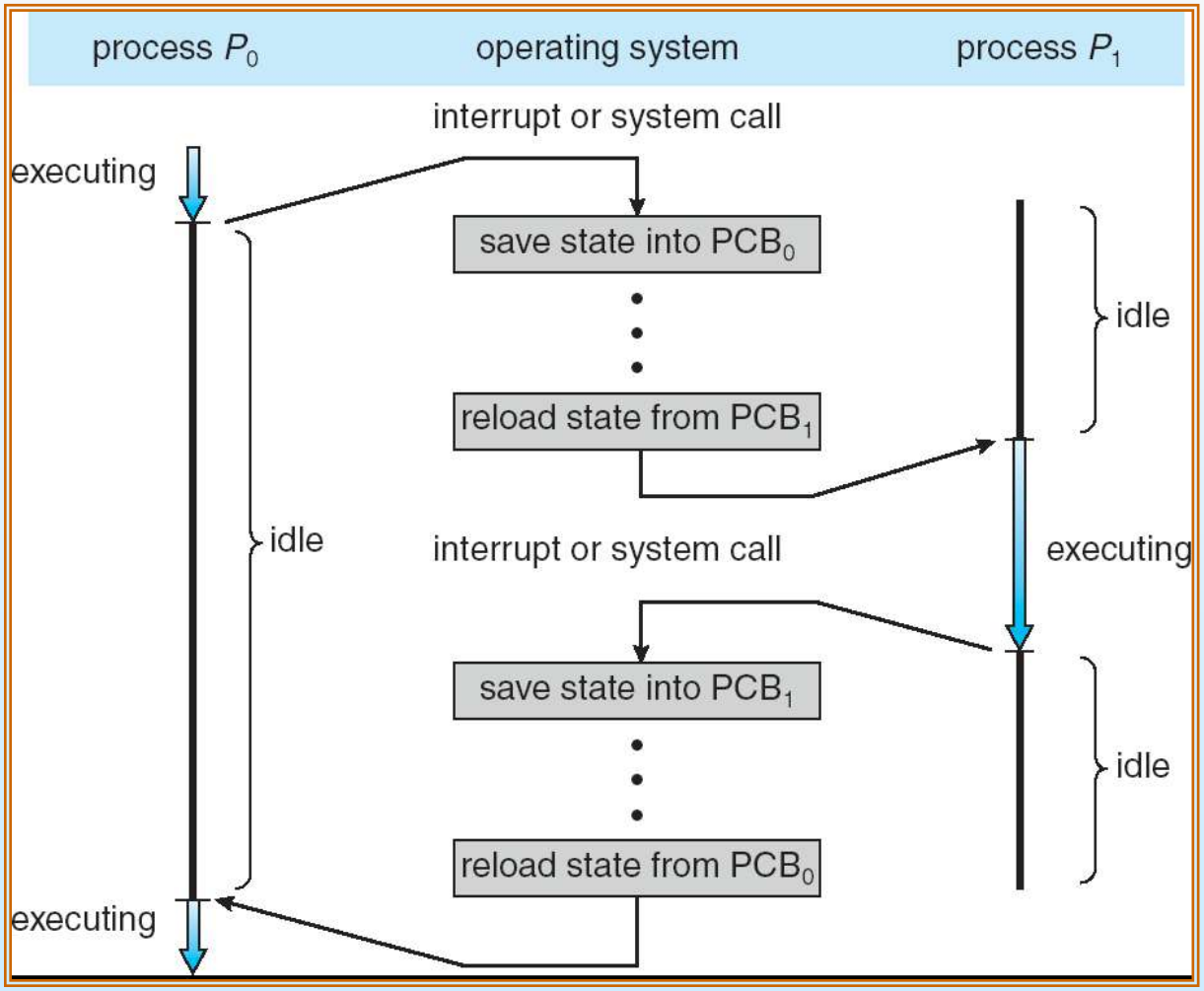
Process Control Block (PCB)







CPU Switch From Process to Process





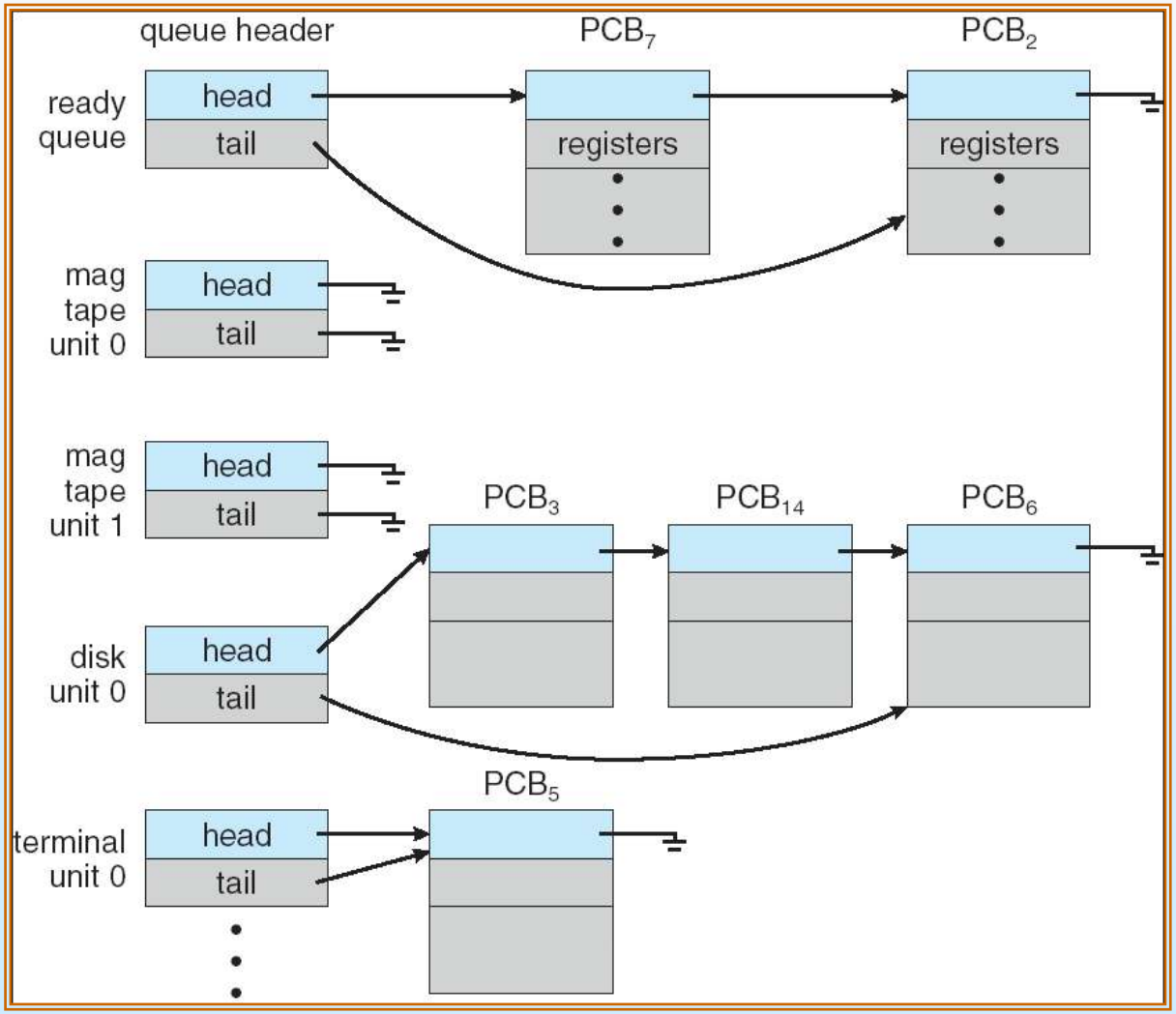


Process Scheduling Queues

- **Job queue** – set of all processes in the system, stored in **job pool** (in the hard disk)
- **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
- **Device queues** – set of processes waiting for an I/O device
- Processes migrate among the various queues
- Each queue is stored as a linked list. For example the ready queue header contains pointers to the first and final PCBs in the list. Eache PCB includes a pointer field that points to the next PCB in the ready queue (see the next slide).

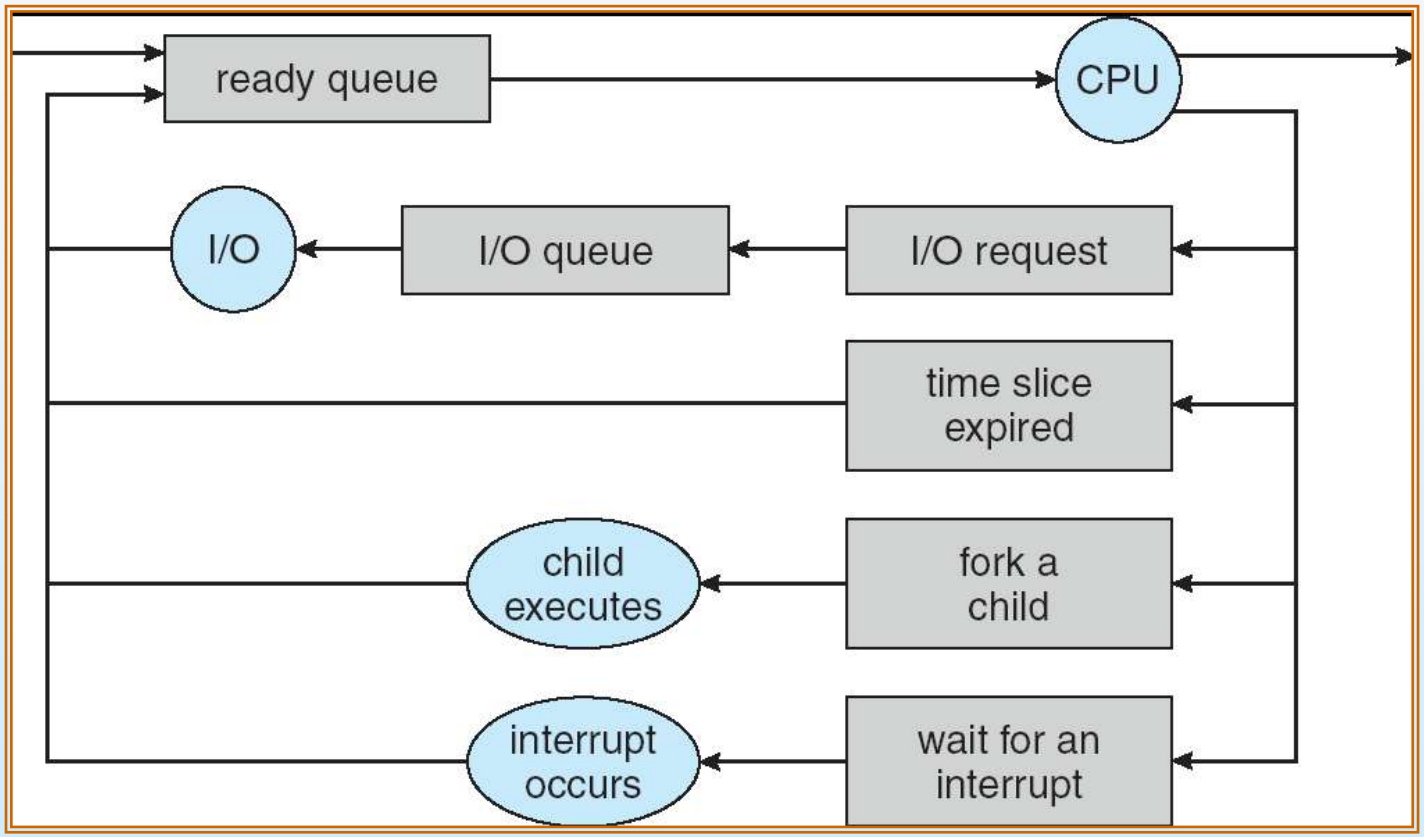


Ready Queue And Various I/O Device Queues





Representation of Process Scheduling







Schedulers

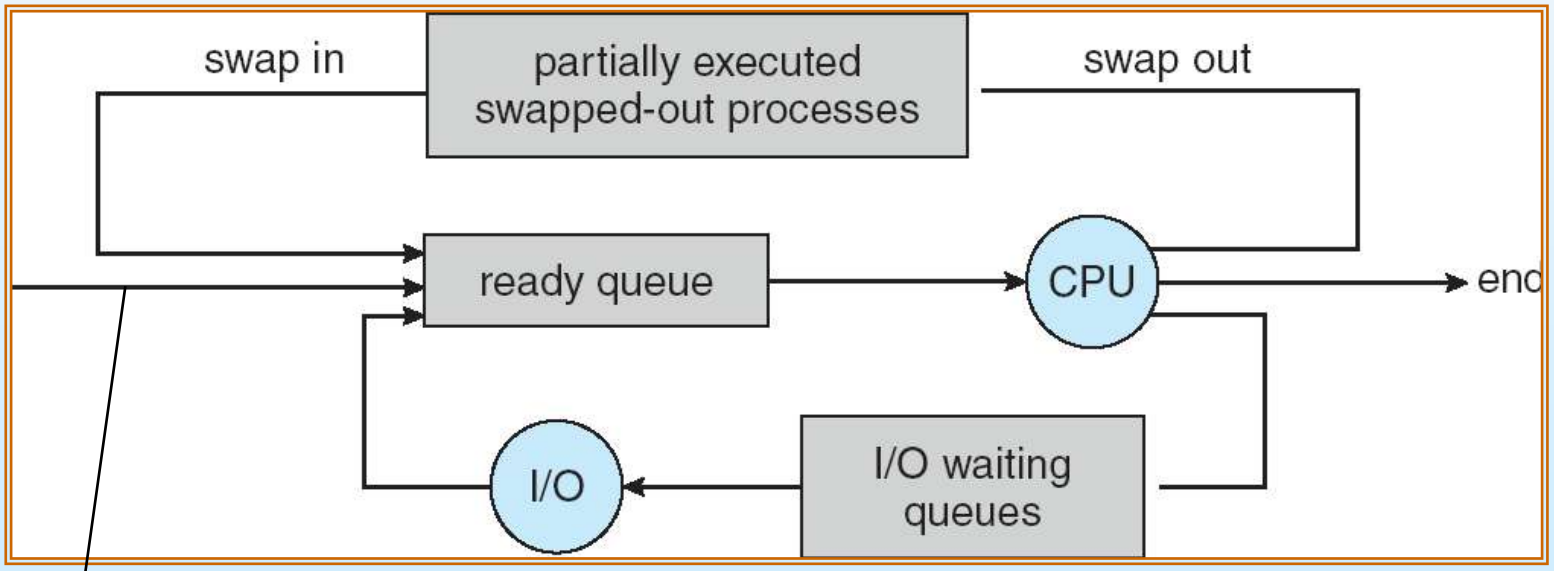
- A process migrate among the various scheduling queues throughout its lifetime.
- **Long-term scheduler** (or job scheduler) – selects which processes should be brought into the ready queue in MM from the job pool in hard disk. It defines the degree of multiprogramming (the number of processes in MM).
  - If the degree of multiprogramming is stable, then the average rate of process creation must equal to the average departure rate of processes leaving the system).
- **Short-term scheduler** (or CPU scheduler) – selects which process in the ready queue should be executed next and allocates CPU
- The main difference between long and short scheduler is the time.
  - Short-term scheduler: should switch between processes once every 100 millisecond.
  - Long-term scheduler: it could take minutes to bring new job (process) form the hard disk to MM.





Addition of Medium Term Scheduling to the queuing diagram (the medium term scheduler use the term swap)

The idea behind medium term scheduling is to control (reduce\increase the number of processes in the ready queue to reduce\increase the degree of multiprogramming) the degree of multiprogramming in time-sharing systems.



Processes coming from the job pool.





Schedulers (Cont.)

- Short-term scheduler is invoked very frequently (milliseconds) ⇒ (must be fast)
- Long-term scheduler is invoked very infrequently (seconds, minutes) ⇒ (may be slow)
- The long-term scheduler controls the *degree of multiprogramming*
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
    - What will happen if all processes are **I/O-bound process** ?
      - Ready queue will be empty.
    - What will happen if all processes are **CPU-bound process** ?
      - I/O waiting queue will be empty.
  - So schedulers should choose a good mix from **I/O-bound process** and **CPU-bound process! Why?**





Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process (***this takes time***)
- **Context-switch time is overhead; the system does no useful work while switching**
- Time dependent on hardware support





Operations on processes

- **Two main operations:**
  - Creation
  - Termination.





Process Creation

- Parent process create children processes, which, in turn create other processes, forming a tree of processes
- When a process create a new process, two possibilities exist in terms of resource sharing
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- When a process create a new process, two possibilities exist in terms of execution:
  - Parent and children execute concurrently
  - Parent waits until children terminate





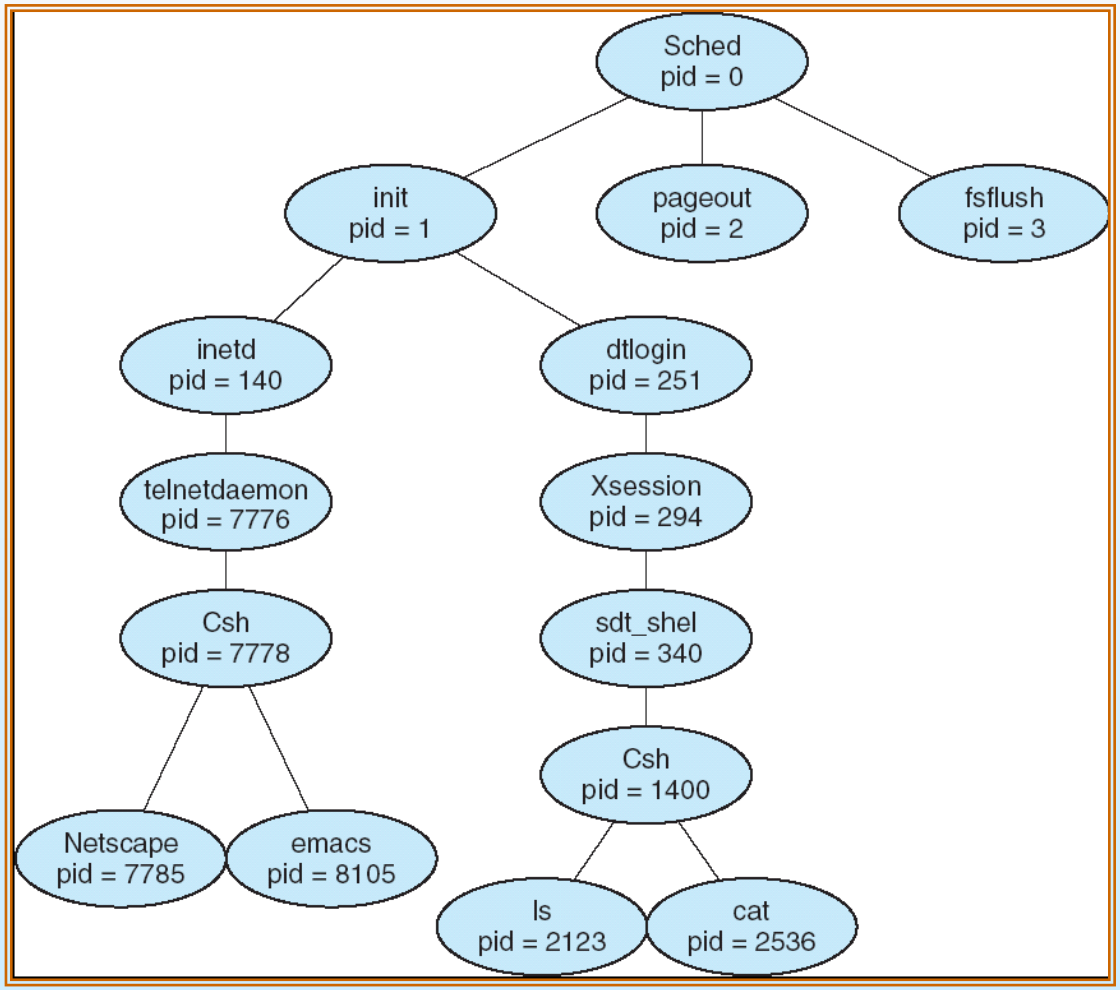
Process Creation (Cont.)

- When a process create a new process, two possibilities exist in terms of address space of the new process:
  - Child duplicate of parent (it has the same program and data as the parent)
  - Child has a program loaded into it





A tree of processes on a typical Solaris







Process Termination

- Process executes last statement and asks the operating system to delete it (**exit**)
  - Output data from child to parent (via **wait**)
  - Process' resources are deallocated by operating system
- Parent may terminate execution of children processes (**abort**)
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - If parent is exiting
    - Some operating system do not allow child to continue if its parent terminates
      - All children terminated - *cascading termination*
        - *cascading termination: is started usually by the OS when a parent terminated. This allows the OS to terminate the childrens.*





Cooperating Processes

- **Cooperating processes require an interprocess communication mechanism (IPC) that allow them to exchange data.**
- **There are two models of IPC: (see the slide 25)**
  - **Shared memory: faster. For large amount of data.**
  - **Message passing: for small amount of data. slower.**
- **Independent** process cannot affect or be affected by the execution of another process (no need for cooperation because it dose not share data with other processes).
- **Cooperating** process can affect or be affected by the execution of another process





Cooperating Processes

- **Advantages of process cooperation**
  - Information sharing (two processes share the same file, is faster than if each process read its own copy from the file, it can be loaded once for both processes).
  - Computation speed-up (dividing the process to sub-processes that executed concurrently will speed up the computations)
  - Modularity (creating a system in a modular fashion as discussed in chapter 2, where the system functions are divided into separate processes)
  - Convenience (a user may execute many tasks at the same time, printing, watching movie, etc..)



Communications Models

